

Question 1.

Why must a checked exception be caught or declared while an unchecked exception does not require either

Checked exceptions must be caught or declared because they represent conditions that a program is expected to handle or recover from. Java enforces this rule at compile time to ensure these conditions are addressed. Unchecked exceptions usually result from programming errors, which should be fixed in the code, so the compiler does not require them to be caught or declared.

Question 2:

Why must a catch block for a subclass come before a catch block for its superclass

A catch block for a subclass must come before a catch block for its superclass because Java executes the first matching catch block. A superclass catch block also catches exceptions of its subclasses. If it comes first, the subclass catch block becomes unreachable, causing a compile-time error.

Question 3.

What happens to an AutoCloseable resource when an exception occurs inside a try with resources block

When an exception occurs inside a try-with-resources block, Java automatically calls `close()` on each successfully initialized resource, in reverse order of initialization. The original exception is propagated. If closing a resource also throws an exception, that exception is recorded as a suppressed exception.

Question 4:

How do throw and throws differ in where they are used and what they do

Throw is used inside a method or block to explicitly throw an exception object.

Throws is used in a method declaration to specify the exception types that the method may throw to its caller.

Throw throws one exception object at a time, while throws can declare multiple exception types.

Question 5.

Why can returning a value from a finally block hide an exception

Returning a value from a finally block overrides any pending exception from the try or catch block. The method returns the value instead of propagating the exception, hiding it from the caller. Therefore, return statements should be avoided in finally blocks.

Question 6:

How can an enum use a field and a method to store and return a value

An enum can define a field to store a value for each constant. A constructor initializes the field using the value supplied by each constant, and a getter method returns the stored value.

Question 7.

When would you choose an ArrayList instead of a LinkedList

I would choose an ArrayList when frequent access by index is needed and insertions or deletions in the middle are uncommon. It provides $O(1)$ indexed access, uses less memory than a LinkedList, and supports efficient appending with amortized $O(1)$ time.

Question 8.

How do HashSet and LinkedHashSet differ when you iterate over their elements

HashSet does not guarantee any iteration order, while LinkedHashSet maintains insertion order. When iterating over a LinkedHashSet, elements are returned in the order they were added.

Question 9:**How do HashMap and TreeMap differ in the order of their keys**

HashMap does not guarantee any order for its keys. TreeMap keeps its keys sorted according to their natural order or a custom Comparator.

Question 10:**Why is an Iterator useful when removing elements from a collection during iteration**

An Iterator allows elements to be safely removed during iteration using its `remove()` method, provided removal is supported. It maintains the iteration state and avoids the `ConcurrentModificationException` that can occur when modifying the collection directly during iteration.